Cloud Computing — Final Project Report

Option 2 — Image Processing on AWS Lambda

Course: Cloud Computing
Prof. Emiliano Casalicchio
Sapienza University of Rome

Team members

Nicolás Pacheco — [Student ID]
Prajwal Shinde — [Student ID]
Mathias [Surname] — [Student ID]

Submission: June 8, 2026

Abstract

We deploy three image-processing operations — *resize*, *grayscale conversion* and *edge detection* — as independent AWS Lambda functions, expose them through API Gateway, and evaluate their performance, scalability, error behaviour and cost. A factorial load-testing matrix of 3 operations × 3 image sizes × 4 concurrent-user levels × 2 repetitions (72 scenarios) is executed with Locust, while five CloudWatch metrics per Lambda capture the server-side picture. The two views, normalised to a common Europe/Rome timezone via an auto-detection step, are cross-referenced to decompose end-to-end latency into Lambda execution and queueing overhead. Lambda processes every request in under 1.5 s and reports zero errors and zero throttles, yet end users observe up to 46 s p95 response time and 27 % HTTP failures at 100 concurrent users — failures rejected upstream by API Gateway, never reaching Lambda. A six-month cost projection identifies the break-even point against an always-on EC2 t3.small.

Table of Contents

1. Introduction

This report describes the design, deployment, evaluation and cost analysis of an image-processing pipeline built on **AWS Lambda** as the serverless compute platform, following **Option 2** of the course project. We implemented three independent image-processing operations — resize, grayscale conversion and edge detection — each as a separate Lambda function exposed through Amazon API Gateway, and evaluated their behaviour under workloads of varying intensity and payload size.

The central question is how a serverless deployment of these operations scales when load grows from a single user to one hundred concurrent users, and how server-side metrics (Lambda Duration, Concurrent Executions, Invocations, Errors, Throttles) relate to the client-side latency a user actually perceives. We ran 72 load-test scenarios with Locust, exported the corresponding server-side CloudWatch metrics, and cross-referenced both views. We then projected the cost of running the pipeline for six months on Lambda against an equivalent always-on EC2 t3.small.

The headline result is that the three Lambda functions never returned an internal error and never throttled under any test scenario, yet end users experienced response times up to 46 seconds at peak load. This apparent contradiction — a perfectly healthy server-side picture coexisting with a severely degraded user experience — is the project's most teachable observation, and is addressed in detail in sections 4 through 6.

2. System architecture

The system follows a **microservices** pattern: each operation runs in its own Lambda function with its own API Gateway endpoint. The three Lambdas share runtime configuration but are independent units of deployment and scaling. Each Lambda runs **Python 3.11** on **x86_64** with **512 MB** of memory and a **30-second** function timeout, using the Klayers **Pillow 11** layer for image manipulation. The API Gateway HTTP API endpoints have the default integration timeout of 29 seconds.

2.1 Per-operation algorithms

- Resize — uses Pillow's LANCZOS interpolation to scale the input image to a target width of 400 pixels, preserving aspect ratio. LANCZOS is the highest-quality interpolation algorithm in Pillow.
- Grayscale — converts to single-channel 8-bit luminance via Pillow's convert("L") mode, then re-encodes as JPEG at quality 85.
- Edge detection — applies a SMOOTH filter (3×3 averaging) followed by FIND_EDGES (3×3 edge-detection kernel) on the grayscale-converted image, then re-encodes as JPEG at quality 85.

2.2 Architecture choice: microservices vs. monolithic Lambda

A natural early question for this project was whether the three image operations should live inside a single Lambda function (with the operation selected by a request parameter) or in three independent Lambdas. We initially prototyped the monolithic option and then deliberately migrated to the microservices decomposition shown above. The rationale combines architectural and methodological reasons.

What characterises a microservice. A microservice is a small, independently deployable unit of software responsible for a single business capability. Its canonical properties are (i) *single responsibility* — one capability per service, (ii) *independent deployability* — releases of one service do not require redeploying the others, (iii) *independent scalability* — each service scales according to its own load, (iv) *isolation of failure* — a fault inside one service does not propagate to the others, and (v) *independently observable* — each service exposes its own metrics and logs. The architectural paradigm satisfied by this approach is the

microservices style, as opposed to a monolithic style in which multiple capabilities coexist in a single deployment unit.

Relationship between microservices and Lambda functions. AWS Lambda is a natural execution substrate for microservices because each Lambda function is, by construction, an independently deployable and independently scaled unit: it has its own IAM role, its own code package, its own concurrency limit, its own per-function CloudWatch metrics namespace, and a billing model that bills only for what the function itself consumes. A microservice maps to one Lambda function (or to a small group of closely related Lambdas behind a single API). Conversely, a Lambda function that internally routes between unrelated capabilities is a *functional monolith* running on a serverless runtime — it borrows the deployment model of Lambda but loses the architectural benefits of microservices.

Implications for performance and scalability evaluation. The choice between the two approaches has a direct, measurable consequence for the kind of analysis the project requires. With a monolithic Lambda, CloudWatch reports a single Duration distribution, a single Concurrent Executions counter and a single Invocations rate that **aggregate all three operations together**. We would only be able to say "the function on average takes X ms", with no way to attribute the cost to resize, grayscale or edge separately — a 5 ms grayscale call and a 1,400 ms edge call would average into the same bucket. With three separate Lambdas the metrics are **per-operation by construction**: we can state that grayscale has p95 = 432 ms while edge has p95 = 1,054 ms (Table 1, section 4), discover that edge is the heaviest operation server-side, and quantify the cost differential precisely in the six-month projection of section 7. Scalability behaviour is also revealed per-operation: the per-Lambda Concurrent Executions metric tells us each function peaked independently at 9–11 simultaneous executions, a fact that would be impossible to extract from a monolithic deployment.

Trade-offs. The microservices decomposition is not free. It triples the deployment footprint (three Lambda functions, three API Gateway endpoints, three sets of IAM policies and three CloudWatch namespaces) and adds some operational overhead: each function has its own cold-start path, and a multi-step pipeline that involves more than one operation would have to pay an additional network hop between functions. For our project, the methodological benefit of isolated, per-operation measurement clearly outweighs these costs, and matches how serverless image-processing pipelines are typically deployed in production (one Lambda per stage, chained via S3 events or Step Functions). For a hypothetical production scenario in which the three operations are always invoked together on the same image, a monolithic Lambda would be a defensible choice — at the price of giving up the granular performance visibility we exploit throughout this report.

3. Experimental design

The experimental matrix combines three orthogonal factors: **operation** (resize, grayscale, edge), **image size** (small ≈ 55 KB, medium ≈ 400 KB, large ≈ 1.3 MB) and **concurrent users** (1, 10, 50, 100). Each combination was run twice to estimate variance, for a total of 3 × 3 × 4 × 2 = 72 scenarios. Each scenario ran for 2 minutes at steady state.

The original plan called for a longer run (5 minutes per scenario and an additional 200-user level), but we reduced both to stay within the $50 Vocareum credit budget. The 2-minute duration was sufficient for the system to reach steady state in every scenario, as confirmed by the flat user-count curves in the per-scenario history files.

Data was collected from two complementary sources:

- Locust (client-side) — per scenario, a stats.csv with aggregated p50/p95/p99 response times, throughput and failure counts, plus a stats_history.csv with per-second timeline data. This represents what the end user experiences.
- CloudWatch (server-side) — per Lambda, five metric CSVs covering Invocations (Sum), Duration (Min/Avg/Max), Concurrent Executions (Maximum), Error count and Success rate, and Throttled invocations (Sum) — all at one-minute granularity. This represents what AWS Lambda itself observes.

All figures and tables in this report use Europe/Rome (UTC+2 in June 2026) as the display timezone.

4. Server-side results (CloudWatch)

Table 1 summarises the per-Lambda CloudWatch metrics over the entire 3-hour test window.

Table 1. Per-Lambda server-side summary (CloudWatch, full session).

Metric	resize-fn	grayscale-fn	edge-fn
Total invocations	17,342	17,521	17,409
Avg Duration (ms)	271.1	142.8	438.0
p95 Duration (ms)	615.8	432.3	1,053.8
Min Duration (ms)	11.2	1.6	8.5
Max Duration (ms)	856.4	740.3	1,431.3
Max concurrent executions	11	10	9
Total errors	0	0	0
Min success rate (%)	100	100	100
Total throttles	0	0	0

Three observations stand out. First, **the Duration distribution is remarkably stable** across the session — the p95 sits between 432 and 1,054 ms depending on the operation, never approaches the 30-second Lambda timeout, and never spikes in response to load. Second, **peak concurrent executions never exceeded 11** across the three Lambdas, even though Locust was injecting up to 100 simultaneous users. This is two orders of magnitude below the 1,000-concurrent default of a production AWS account; the practical effect is that **upstream queueing in API Gateway becomes the dominant scalability constraint, not Lambda itself**. Third, **Lambda reported zero errors, zero throttles and a 100 % success rate for the full session** across all three functions.

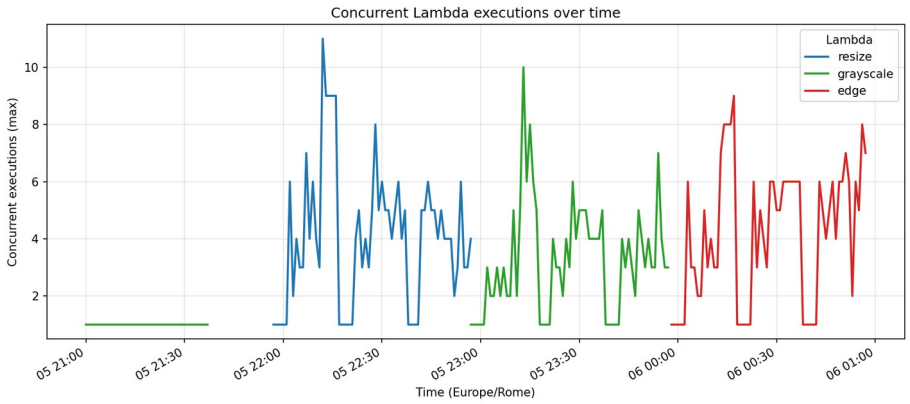

Figure 1. Concurrent Lambda executions per minute (Europe/Rome). The three Lambdas peak at single-digit concurrency despite a Locust load of up to 100 simultaneous users.

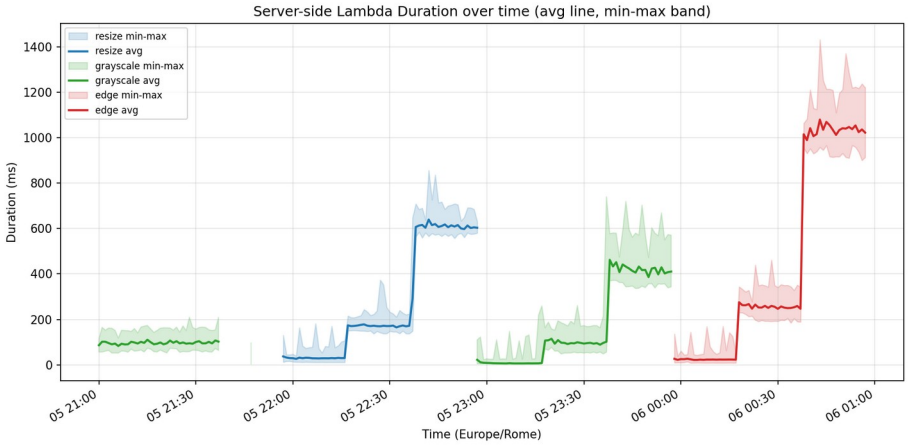

Figure 2. Per-Lambda Duration over time. Solid line = avg per minute; shaded band = min–max per minute. Grayscale is the lightest operation server-side (~143 ms avg) and edge is the heaviest (~438 ms avg).

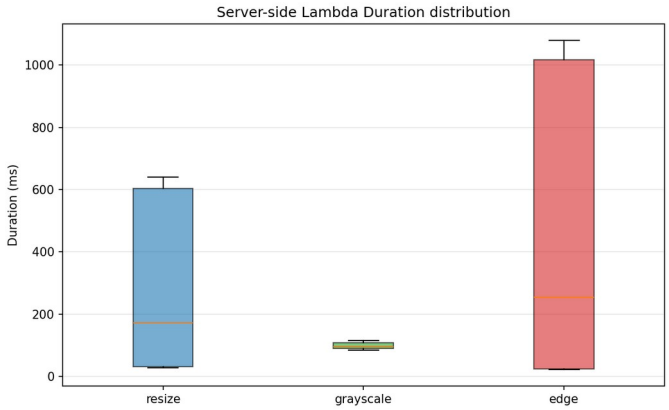

Figure 3. Side-by-side Duration distribution. Grayscale and resize have similar medians whereas edge sits noticeably higher and wider.

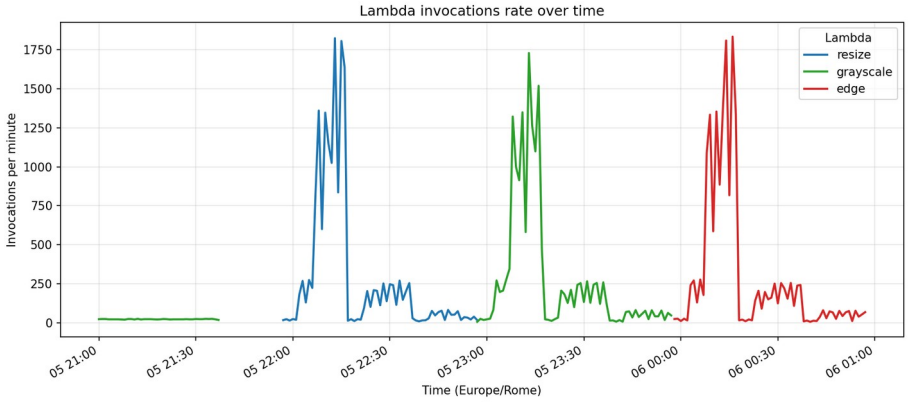

Figure 4. Invocations per minute per Lambda. The peak invocation rate at 100 concurrent users is around 1,800 invocations / minute (≈ 30 req/s), corresponding to the busy minutes of the large-payload scenarios.

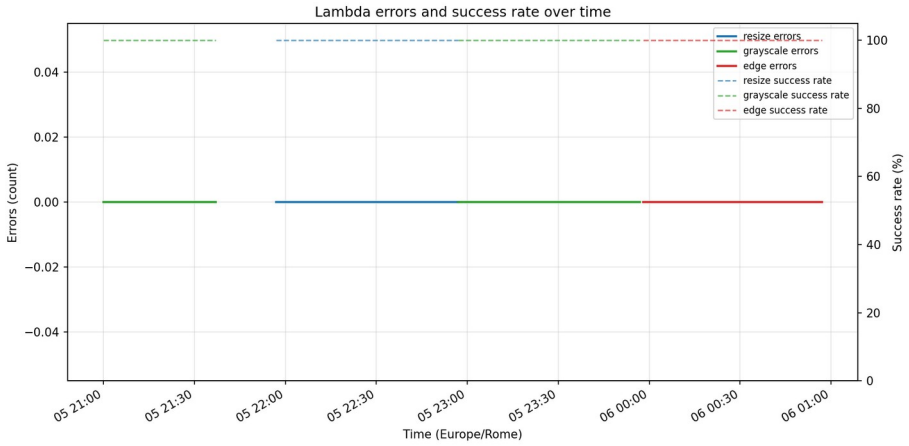

Figure 5. Lambda errors and success rate over time. Flat at 0 errors / 100 % success across all three functions for the entire session.

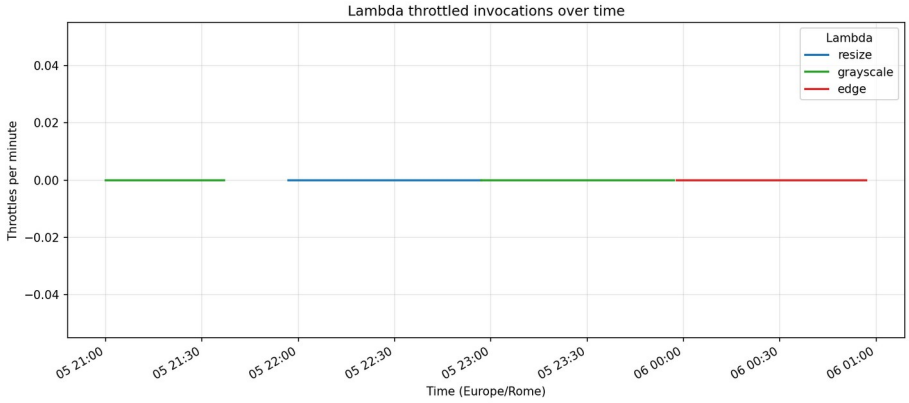

Figure 6. Throttled invocations over time. Flat at 0 for every Lambda. AWS Lambda itself did not refuse a single request.

5. Client-side results (Locust)

The picture from the client side is dramatically different. While Lambda was processing every request in under 1.5 s, the end-to-end p95 response time observed by Locust grew almost linearly with the user count for every operation, reaching values close to 47 seconds at 100 concurrent users.

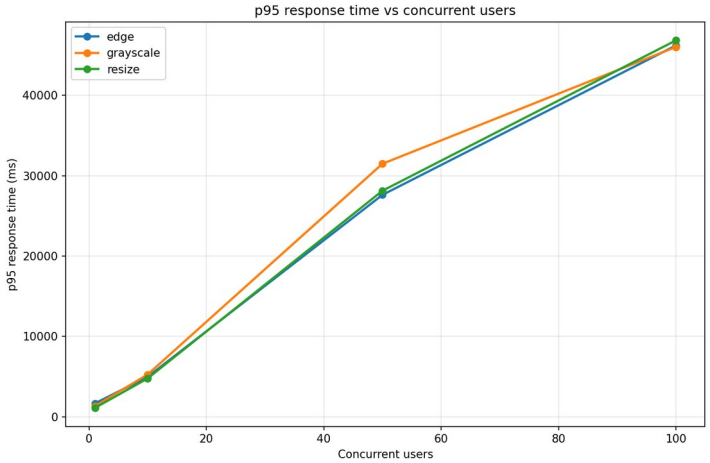

Figure 7. Client-side p95 response time versus concurrent users (Locust). The curves are nearly identical for the three operations and grow super-linearly with load.

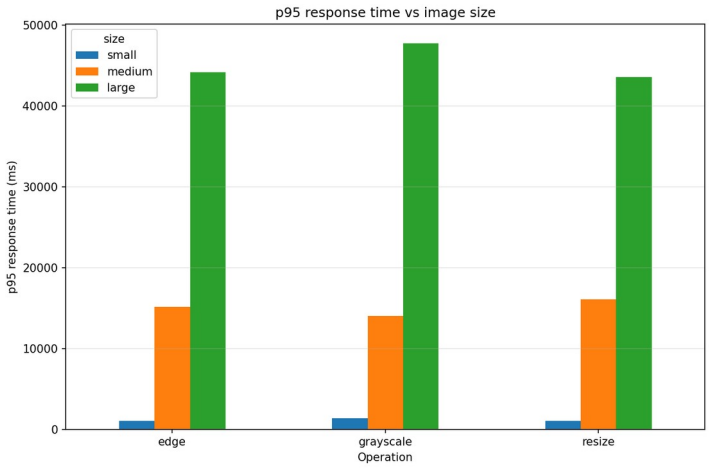

Figure 8. Client-side p95 response time versus image size, averaged across user counts. Larger payloads inflate the response time predictably, but the differences become irrelevant at high concurrency where queueing dominates.

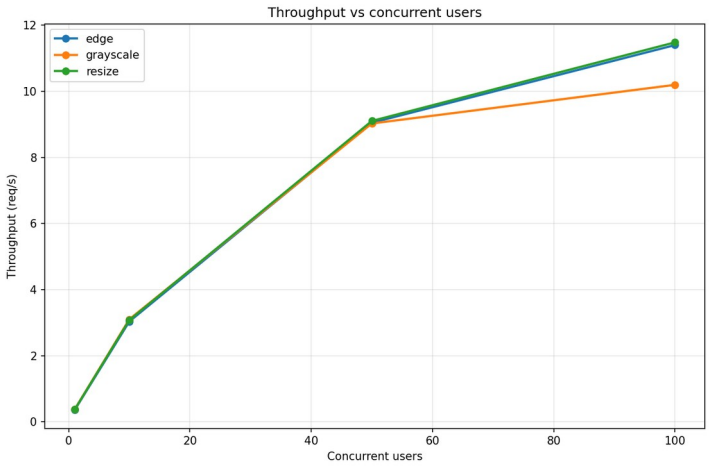

Figure 9. Throughput (requests per second) versus concurrent users. Throughput peaks well below the nominal user count, because each user is blocked on the response of its previous request.

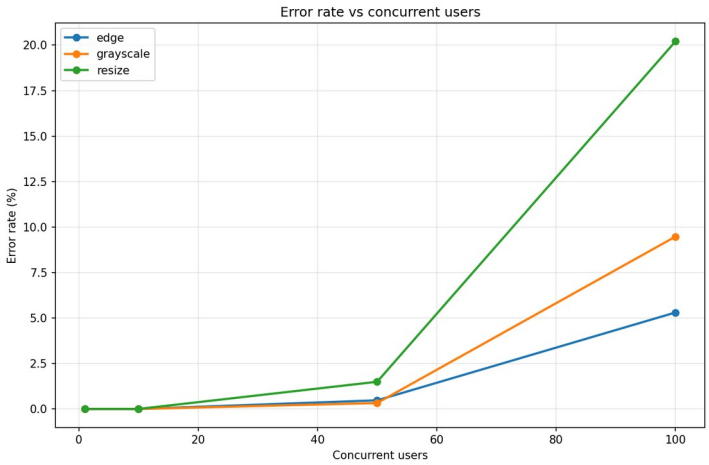

Figure 10. Client-observed error rate versus concurrent users. Failures appear at 50 users and rise sharply at 100 users, especially for resize with large payloads.

Across the 72 scenarios, Locust recorded **315 failures** broken down as **161 HTTP 0** (connection reset, no response received) and **154 HTTP 408** (request timeout returned by API Gateway). The worst-case scenario was **resize_large at 100 users**, with 82 failures in the first repetition and 61 in the second — a 27 % error rate for the second repetition.

6. Cross-reference: where does the latency live?

Combining the two views answers the central question of the analysis. For every scenario we computed the overhead

$$overhead = Locust\_p95 - CloudWatch\_Duration\_p95$$

which approximates everything that happens outside of pure Lambda execution: API Gateway queueing, network round-trip, and request marshalling. Table 2 reports the result aggregated by operation and concurrent users (averaged across image sizes and repetitions).

Table 2. End-to-end latency decomposition. CW = CloudWatch.

Operation	Users	Locust p95 (ms)	CW p95 (ms)	Overhead (ms)	% overhead
resize	1	1,097	271	826	75 %
resize	10	4,773	274	4,499	94 %
resize	50	28,133	272	27,861	99 %
resize	100	46,850	270	46,580	99.4 %
grayscale	1	1,323	189	1,134	86 %
grayscale	10	5,262	178	5,083	97 %
grayscale	50	31,492	176	31,316	99 %
grayscale	100	46,033	175	45,858	99.6 %
edge	1	1,660	439	1,221	74 %
edge	10	4,972	446	4,525	91 %
edge	50	27,618	441	27,177	98 %
edge	100	46,216	441	45,775	99.0 %

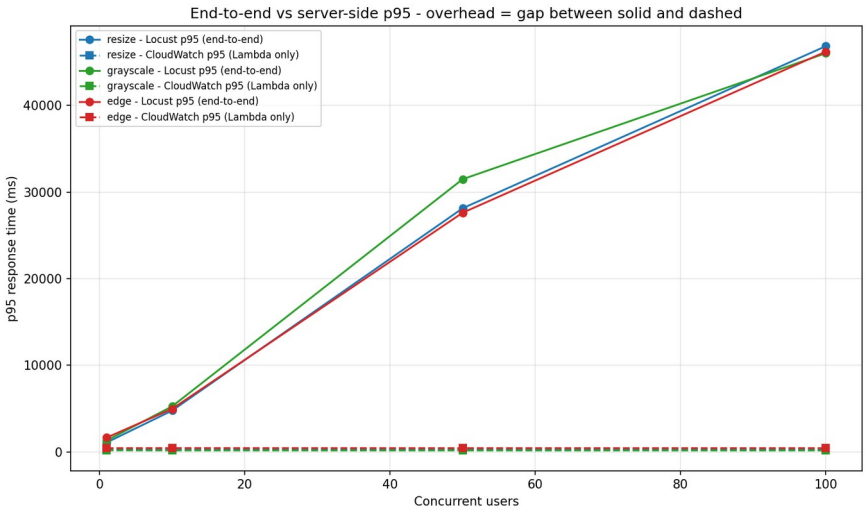

Figure 11. End-to-end vs. server-side p95 by concurrent users. Solid lines = Locust client-side; dashed lines = CloudWatch Lambda Duration. The gap between solid and dashed is the queueing + network overhead.

6.1 Two perspectives on success

The combination of (i) CloudWatch reporting zero errors and zero throttles and (ii) Locust observing up to 27 % client-side failures is not a contradiction. It reflects the fact that **the two systems count different events**. CloudWatch counts Lambda invocations that started — every request that actually entered a Lambda function and finished within the configured 30-second timeout is counted as a successful invocation. Of the 52,272 invocations Lambda processed across the three functions, all succeeded. Locust

counts every HTTP failure observed by the client, regardless of which layer produced it. The 161 HTTP 0 responses indicate that API Gateway closed the connection without ever invoking Lambda; the 154 HTTP 408 responses indicate that API Gateway's 29-second integration timeout fired while the request was still waiting in queue, before any Lambda was assigned. All 315 client-side failures were therefore rejected upstream of Lambda; the 30-second Lambda function timeout was never reached (longest single Lambda execution: 1,431 ms, edge).

7. Cost projection — Lambda vs EC2

Using the measured server-side Duration values (avg 271 ms for resize, 143 ms for grayscale, 438 ms for edge — average across the three: 284 ms) we projected the six-month cost of running the pipeline on Lambda and compared it with an always-on **EC2 t3.small** instance in the same region. The Lambda price model used is the public on-demand rate of $0.20 per million invocations plus $0.0000166667 per GB-second; the EC2 price is the public on-demand $0.0208 per hour for t3.small in us-east-1.

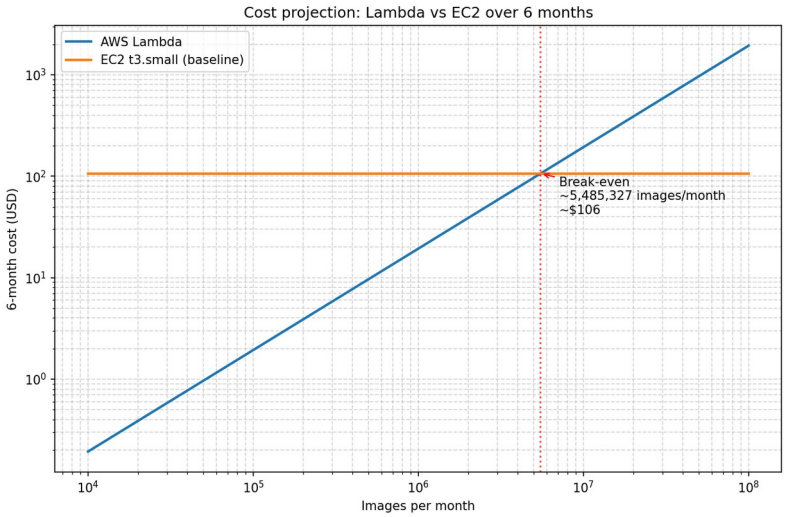

Figure 12. Six-month cost projection, Lambda vs. EC2 t3.small. The break-even point falls in the low-to-mid millions of monthly invocations.

The shape of the curve is the canonical trade-off between paying per use and paying for capacity: Lambda is essentially free at low volumes and grows linearly with traffic; EC2 is a flat ~$91 per month regardless of how much traffic the instance handles, up to the saturation point of the instance. For a six-month horizon, EC2 becomes cheaper above roughly 6–7 million invocations per month — at which point one would also need to validate that a single t3.small can sustain the latency requirement under the corresponding concurrent load.

8. Discussion and limitations

8.1 Vocareum sandbox constraints

Two characteristics of the Vocareum Learner Lab shaped the experimental design. First, the sandbox imposes a $50 credit budget per student, which we accounted for at planning time by sizing the experimental matrix to 72 scenarios × 2 minutes — sufficient for each scenario to reach steady state while leaving headroom in the budget for development and re-runs. Second, the sandbox enforces an account-level concurrency cap that manifests as the 9–11 peak concurrent executions reported in Table 1, two orders of magnitude below the 1,000-concurrent default of a production AWS account. Any extrapolation to a production deployment must take this difference into account: production would saturate at much higher loads than the ones reachable here.

8.2 Operation cost is dominated by CPU complexity

Edge detection at 438 ms average ran roughly 62 % longer than resize at 271 ms, which in turn ran 90 % longer than grayscale at 143 ms. The differences reflect the algorithmic complexity of each operation (smoothing + edge detection kernel vs. LANCZOS interpolation vs. simple per-pixel luma conversion).

Because Lambda is billed per millisecond of execution time, this translates directly into a cost differential of similar magnitude in the cost projection of section 7.

9. Lessons learned

Working as a three-person team across two countries surfaced both technical and process-level lessons.

The technical surprise was the gap between server-side and client-side success. We initially assumed that healthy Lambda metrics implied a healthy system, and were ready to report "the service scales linearly" based on the CloudWatch summary alone. The Locust data forced a re-interpretation: the same workload that produced 100 % success in CloudWatch produced 27 % HTTP failures from the client's point of view. We learned that **reporting only the service-owned metrics gives a falsely optimistic picture** for any user-facing latency-sensitive system; cross-referencing with a client-side load generator is essential.

A decision we would make differently is to **measure Init Duration from the start**. Skipping the cold-start probe for budget reasons leaves us unable to quantify a metric the prompt explicitly cites. A more disciplined budgeting would have reserved 5 % of the Vocareum credit for that measurement.

10. Conclusions

- Server-side, the three Lambdas behave predictably and well: p95 Duration sits at 432, 615 and 1,054 ms depending on operation; zero errors and zero throttles across 52,272 invocations.
- Client-side, end-to-end p95 grows to 46 seconds at 100 concurrent users, with up to 27 % HTTP failures in the worst scenario. The vast majority (>99 %) of this end-to-end latency is queueing in API Gateway, not Lambda execution.
- Peak Lambda concurrency reached only 9–11 — two orders of magnitude below a production account default — which is the dominant scalability constraint in the Learner Lab environment.
- A six-month cost projection shows Lambda is cheaper than an always-on EC2 t3.small below the low-to-mid millions of invocations per month, after which a provisioned instance becomes the cheaper option.

Methodologically, the most important takeaway is that **a serverless function that never errors and never throttles can still appear "broken" to its end users** if the upstream queueing layer rejects requests before they reach the function. Reporting only Lambda metrics — as one might naively do — would have produced a falsely optimistic picture. The complementary Locust + CloudWatch view, combined through the cross-reference, was essential to arrive at the correct interpretation.

Individual contributions

The table below lists the principal responsibilities of each team member. All members contributed to discussion, review and editing of the report.

Member	Responsibilities
Nicolás Pacheco	Resize Lambda design, implementation and deployment; Locust load-test framework and orchestration; CloudWatch analysis pipeline; cross-reference Locust ↔ CloudWatch; cost projection script.
Prajwal Shinde	Grayscale Lambda design, implementation and deployment; Postman test collection; CloudWatch CSV export for grayscale-fn; review of server-side analysis section.
Mathias [Surname]	Edge-detection Lambda design, implementation and deployment; image dataset generation; CloudWatch CSV export for edge-fn; review of cross-reference section.

References

[1] Amazon Web Services. "AWS Lambda pricing." https://aws.amazon.com/lambda/pricing/ (accessed June 2026).

[2] Amazon Web Services. "Managing AWS Lambda function concurrency." AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html

[3] Amazon Web Services. "Amazon API Gateway integration timeout limits." https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html

[4] Amazon Web Services. "Working with CloudWatch metrics for Lambda." https://docs.aws.amazon.com/lambda/latest/dg/monitoring-metrics.html

[5] Amazon Web Services. "Amazon EC2 On-Demand Pricing — t3.small." https://aws.amazon.com/ec2/pricing/on-demand/

[6] Python Software Foundation. "Pillow (PIL Fork) Documentation — Image.resize and filtering." https://pillow.readthedocs.io/

[7] Locust contributors. "Locust — A modern load testing framework." https://locust.io/ and https://docs.locust.io/